# ALGORITHM AVENGERS
# DIY-ReinforcementLearning-ModelFree
# Deep Q Network (DQN)
# - Swapnendu

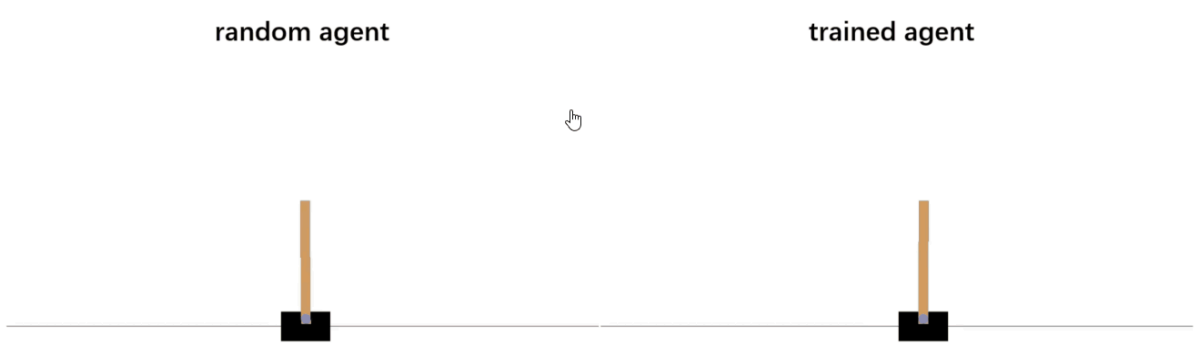
**Pre-requisites: Understanding of Q-learning.**

## Introduction:

Deep Q-Network (DQN) is a powerful algorithm in the field of reinforcement learning. It combines the principles of deep neural networks with Q-learning, enabling agents to learn optimal policies in complex environments.

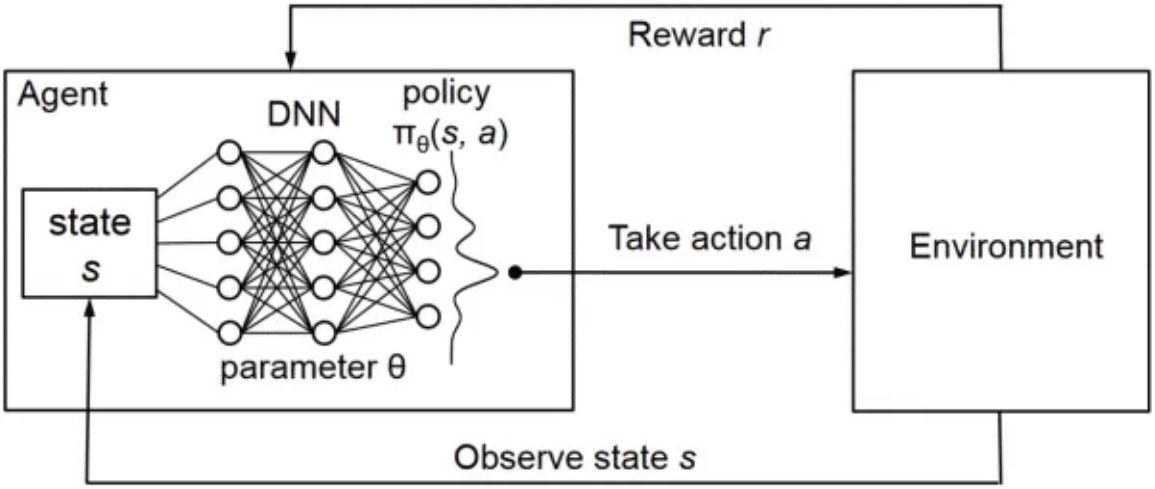
## Problem Statement:

The goal is to train an agent to maintain the balance of a pole on a cart for as long as possible. The agent receives rewards based on its performance and must learn to take actions that maximize these rewards over time.



## Overview of DQN:

DQN is a value-based reinforcement learning algorithm that combines Q-learning with deep neural networks. It uses a neural network to approximate the Q-values for different actions in a given state, enabling the agent to make informed decisions based on its experiences.




## Working of DQN:

Deep Q-Networks (DQN) combine Q-learning with deep neural networks to enable an agent to learn optimal actions in complex environments. The agent interacts with the environment by taking actions based on its current state and receives rewards that indicate the quality of those actions. DQN utilizes a neural network to approximate the Q-values for each action given a state, allowing the agent to generalize knowledge across similar states. During training, the agent stores experiences in a replay memory, which contains tuples of (state, action, reward, next state, done). Periodically, the agent samples random mini-batches from this memory to update the Q-values using the Bellman equation. This helps stabilize training and mitigate issues like correlated experiences. The exploration-exploitation dilemma is managed by gradually decreasing the exploration rate (epsilon) over time, enabling the agent to shift from exploring new actions to exploiting known rewarding actions. Overall, DQN effectively learns optimal policies in environments with high-dimensional state spaces, such as video games or robotic tasks.

# Components of DQN:

## 1. DQNAgent Class

The class encapsulates the entire DQN implementation, including memory management, action selection, model training, and more.

Attributes:

- state_size: The dimensionality of the input state.
- action_size: The number of possible actions.
- memory: A deque to store past experiences.
- gamma: The discount factor for future rewards.
- epsilon: The exploration-exploitation parameter.
- learning_rate: The rate at which the model learns.

## 2. Neural Network:

A feedforward neural network is used to approximate Q-values, which allows the agent to generalize its learning across different states.

## 3. Replay Memory:

A deque is used to store past experiences (state, action, reward, next state, done) to break the correlation between consecutive experiences, improving the stability of training.

## 4. Training Process:

The agent interacts with the environment, collects experiences, and updates its Q-values based on a sampled batch of experiences from memory.

# Equations behind DQN:

Q-Learning Update Rule:

- $Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma max\, Q_{a'}(s',a') - Q(s,a))$

Where:

- $Q(s,a)$: $The\ current\ estimate\ of\ the\ Q - value\ for\ state\ sss\ and\ action\ a$
- $r$: $The\ immediate\ reward\ received\ after\ taking\ action\ a$
- $\gamma$: $The\ discount\ factor$
- $max_{a'}\, Q(s',a')$: $The\ maximum\ predicted\ Q - value\ for\ the\ next\ state\ s'$
- $\alpha$: $The\ learning\ rate,\ controlling\ how\ much\ of\ the\ new\ Q\ influence\ current\ estimate$

Target Value Calculation:

- $target = r\ if\ done$
- $target = r + \gamma max\, Q_{a'}(s',a')\ if\ not\ done$

Discount Factor($\gamma$) :

- $Q(s,a) = r + \gamma\, max_{a'} Q(s',a')$
- Helps the network to prioritize short term rewards compared to long term.

Epsilon (Exploration Rate):

- Either 'Select Random Action' with probability **ε**
- Or 'Select Best Action' with probability **ε-1**
- The value of Epsilon decreases over time, initially the agent prefers to explore but as the game progresses it prefers exploiting the best actions from the memory buffer.

# DQN Algorithm :

## 1. Initialization

- Create an instance of the DQNAgent class with the defined state size and action size.
- Initialize parameters such as epsilon, gamma, learning rate, and the replay memory.

## 2. Environment Setup

- Choose the environment (e.g., CartPole) to train the agent.
- Reset the environment to get the initial state.

## 3. Episode Loop

- For each episode (a complete run from start to finish):
    1. **Reset the Environment**: Obtain the initial state.
    2. **State Reshape**: Reshape the state to match the input format expected by the neural network.
    3. **Time Loop**: For a maximum number of steps in the episode:

        - **Render Environment**: (Optional) Visualize the environment.
        - **Action Selection**: Use the act method to choose an action based on the current state (explore or exploit).
        - **Step the Environment**: Execute the chosen action and observe the next state, reward, and whether the episode is done.
        - **Reward Adjustment**: Modify the reward based on whether the episode has ended (e.g., apply a penalty if done).
        - **State Reshape**: Reshape the next state for model input.
        - **Store Experience**: Use the remember method to save the current experience (state, action, reward, next state, done) in memory.
        - **Transition to Next State**: Update the current state to the next state.
        - **Check Episode Termination**: If done, log the episode score and exit the time loop.

## 4. Experience Replay

- After each episode, if the memory contains enough experiences:
    - **Sample Mini-batch**: Randomly select a batch of experiences from memory.
    - **Q-value Update**: For each experience in the mini-batch:
        - Compute the target value based on the reward and future Q-values (using the Q-learning update rule).
        - Update the predicted Q-values for the current state using the calculated target.
        - Train the neural network on this updated target.

## 5. Epsilon Decay

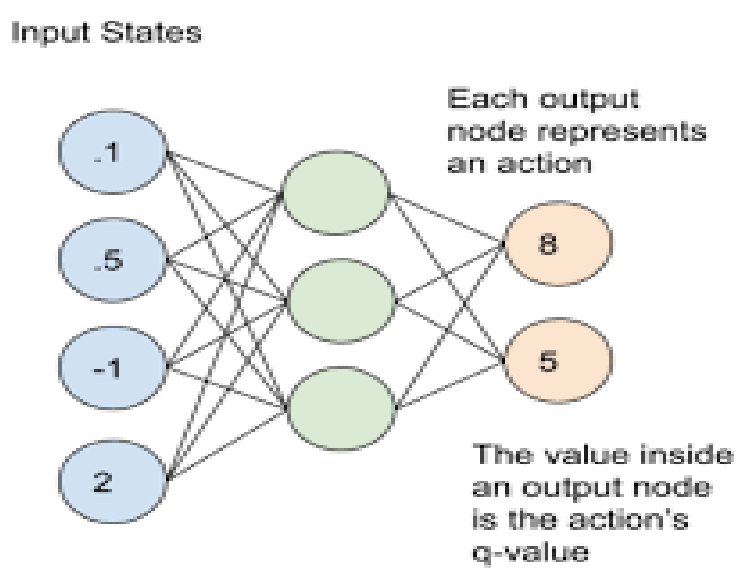
- After each episode, the exploration rate (epsilon) gradually shifts from exploration to exploitation.

## 6. Model Saving

- Periodically save the model weights (e.g., every 100 episodes) to preserve the learned parameters.

## 7. Termination

- The training continues for a specified number of episodes or until a certain performance threshold is reached.



# Hyperparameters:

- State Size: The number of features representing the environment's state. This is the input dimension for the neural network.

- Action Size: The total number of possible actions the agent can take in the environment.

- Memory: The replay buffer (often a deque) that stores past experiences (state, action, reward, next state, done) for training. This helps the agent learn from past experiences instead of relying solely on recent ones.

- Gamma ($\gamma$): The discount factor used to balance immediate and future rewards. A value close to 1 prioritizes future rewards, while a value close to 0 emphasizes immediate rewards.

- Epsilon ($\varepsilon$): The exploration rate that determines the likelihood of taking a random action versus the best-known action. Initially set high for exploration, it decays over time as the agent learns.

- Epsilon Decay: The rate at which epsilon decreases over time. This controls how quickly the agent shifts from exploration to exploitation.

- Epsilon Min: The minimum value epsilon can reach, ensuring that the agent continues to explore occasionally even after training.

- Learning Rate: The step size for updating the weights of the neural network during training. A smaller learning rate leads to more stable but slower learning.

- Batch Size: The number of experiences sampled from the memory during each training update. A larger batch size can lead to more stable gradients.

## Advantages:

- **Generalization**: DQN uses deep learning to generalize learning across states, allowing it to handle high-dimensional state spaces (e.g., pixels in video games) effectively.
- **Experience Replay**: The use of a replay buffer allows the agent to learn from past experiences, reducing the correlation between consecutive experiences and improving learning stability.
- **Sample Efficiency**: By training on mini-batches of experiences, DQN can learn more effectively from a limited number of interactions with the environment.
- **Dueling Architecture**: Variants of DQN (like Dueling DQN) separate the estimation of state values and advantage values, leading to more efficient learning of Q-values.

## Disadvantages:

- **Overestimation Bias**: DQN can suffer from overestimation bias when updating Q-values, particularly if the network architecture or learning rate is not well-tuned.
- **Instability**: DQN training can be unstable due to the nature of deep learning, leading to divergence if hyperparameters are not carefully set.
- **Computational Resources**: DQN requires significant computational resources and time, especially for large networks and environments with complex state representations.
- **Exploration Challenges**: While epsilon-greedy strategies help with exploration, they may not always lead to the best exploration strategy, potentially causing the agent to miss better actions.